Impariamo a ad usare Github come SCM

AA2627

Angelo Gargantini
Ingegneria del Software

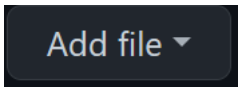
Sviluppiamo un piccolo software – un progetto eclipse maven per una calcolatrice (class Calculator) che fa alcune operazioni tra numeri (somma, etc.)

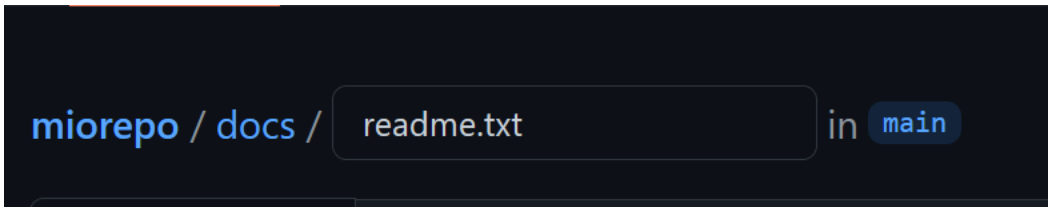
1. Creazione account e formazione dei gruppi

- Crea un account su github se non ce l'hai già
- Crea un gruppo di 2/3/4 persone (ad esempio per ogni fila)

2. Crea repository in remoto

- Crea un solo repository per gruppo (con nome ad esempio "calculator") direttamente sul sito
- Chi lo crea inserisce anche gli altri come membri del progetto
    - Quando lo crei scegli già il tipo di progetto, così ti crea il file .gitignore idoneo
    - Noi usiamo come tipo di progetto "java"
- Nel repository crea una directory docs per i documenti e una directory codice per il codice. Fai queste operazioni sul sito direttamente

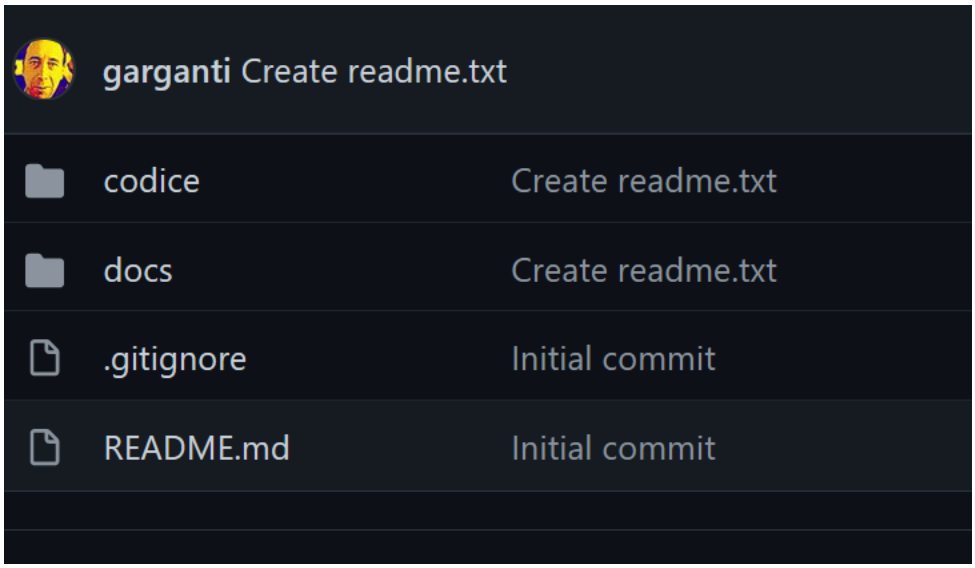
  Usando Add file 

  (meglio creare il file README.md)

  E poi commit new file (in fondo alla pagina)

  ATTENZIONE: non si possono creare directory vuote in github, creiamo quindi dei file tipo reame.md o readme.txt



1. Attenzione: cosa è il .gitignore

Spesso, quando si lavora su un progetto che utilizza Git, è necessario escludere file o directory specifici dall'invio al repository remoto.

Il file .gitignore viene utilizzato per specificare quali file non tracciati devono essere ignorati da Git.

I file ignorati sono generalmente file specifici della piattaforma o file creati automaticamente dai sistemi di compilazione. Alcuni esempi comuni includono:

File di runtime come log, lock, cache o file temporanei.

File con informazioni riservate, come password o chiavi API.

Codice compilato, come .class o .o.

Directory delle dipendenze, come /vendor o /node_modules.

Directory di compilazione, come ad esempio /public, /out o /dist.

File di sistema come .DS_Store o Thumbs.db

File di configurazione di IDE (come .metadata) o dell'editor di testo.

Un file .gitignore è un file di testo semplice in cui ogni riga contiene un modello da ignorare per file o directory.

.gitignore usa schemi di globbing per abbinare nomi di file e caratteri wildcard. Se hai file o directory contenenti un modello wildcard, puoi usare una singola barra rovesciata (\) come carattere di escape.

Commenti

Le righe che iniziano con un segno di cancelletto (#) sono commenti e vengono ignorate. Le righe vuote possono essere utilizzate per migliorare la leggibilità del file e per raggruppare le relative linee di modelli.

Slash

Il simbolo della barra (/) rappresenta un separatore di directory. La barra all'inizio di un modello è relativa alla directory in cui risiede .gitignore.

Se il modello inizia con una barra, corrisponde ai file e alle directory solo nella root del repository.

Se il modello non inizia con una barra, corrisponde a file e directory in qualsiasi directory o sottodirectory.

Se il modello termina con una barra, corrisponde solo alle directory. Quando una directory viene ignorata, vengono ignorati anche tutti i suoi file e le sue sottodirectory.

Nomi di file letterali

Il modello più semplice è un nome file letterale senza caratteri speciali.

Pattern	Example matches
/access.log	access.log
access.log	access.log
	logs/access.log
	var/logs/access.log
build/	build

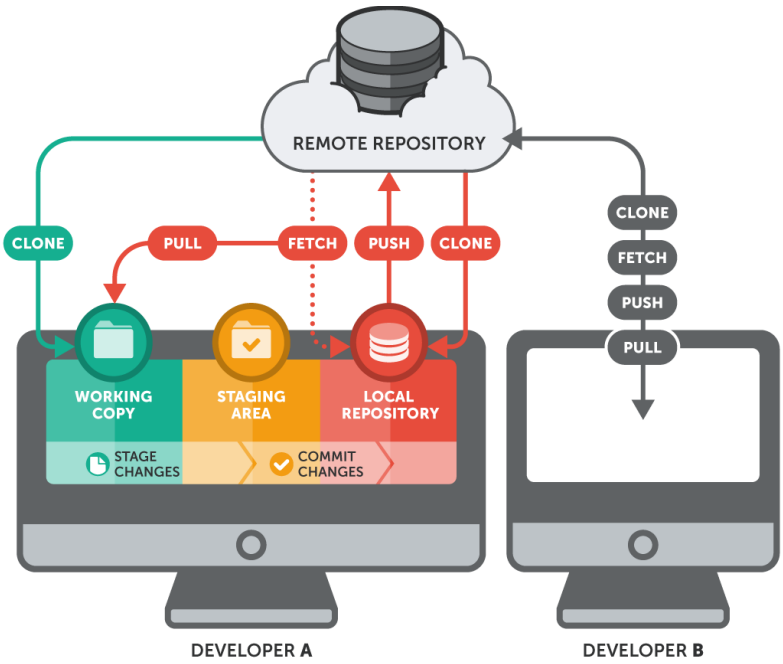
Simboli Wildcard

* - Il simbolo dell'asterisco corrisponde a zero o più caratteri.

**- Due simboli di asterisco adiacenti corrispondono a qualsiasi file o zero o più directory. Quando seguito da una barra (/), corrisponde solo alle directory.

? - Il punto interrogativo corrisponde a qualsiasi singolo carattere.

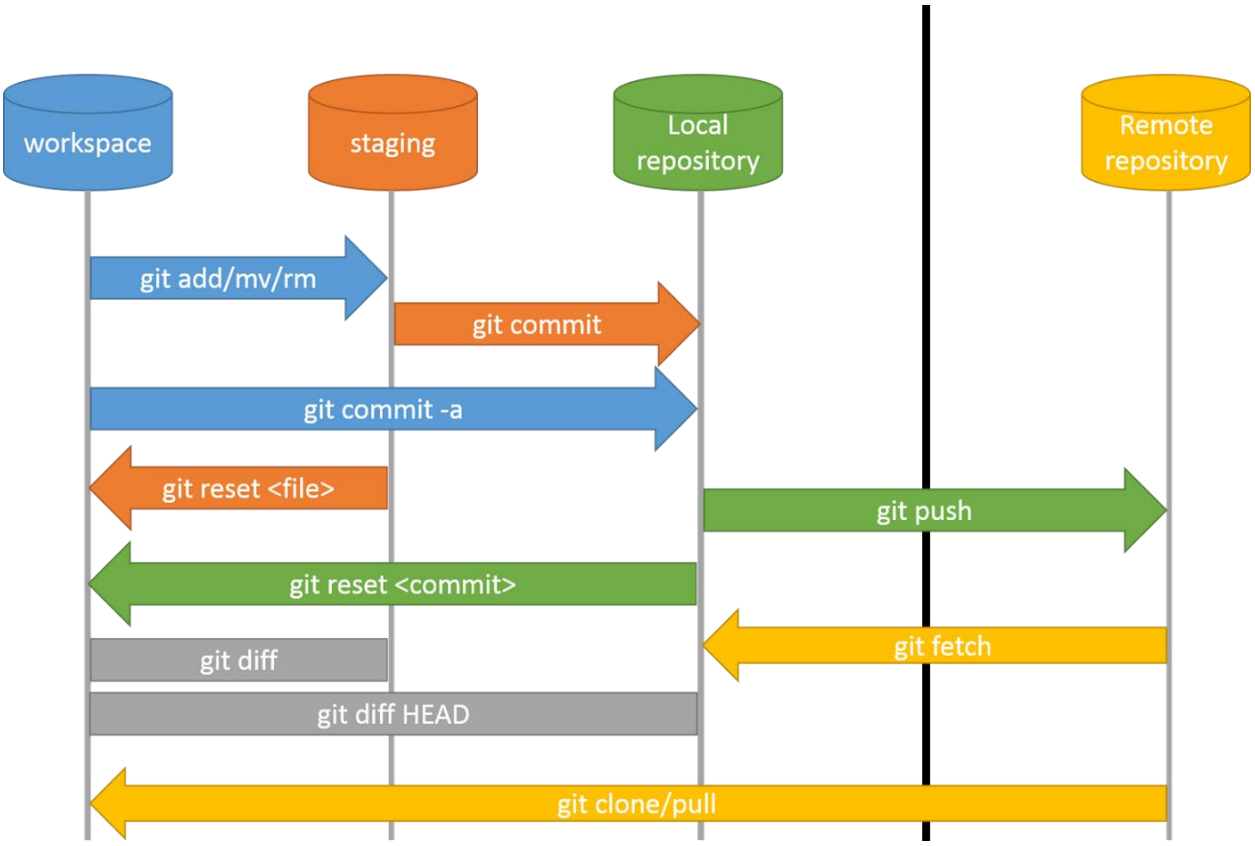
3. Come funziona github



Remote repository: repository remota dove si vanno a finire tutti i file con le informazioni sulle loro versioni (con anche le info su branches, modifiche, etc..)

Localmente (nel .git):

1. **Working copy**: copia attuale dei file su cui si sta lavorando.
2. **Staging area e local repository**: qui vengono passate le informazioni sulle modifiche (changes) che si vogliono inviare al repository remoto



Flusso tipico per inviare una modifica al repository remoto:

ADD: aggiunge un file

COMMIT: rende visibili i cambiamenti sul repository locali

PUSH: manda le modifiche sul repo remoto

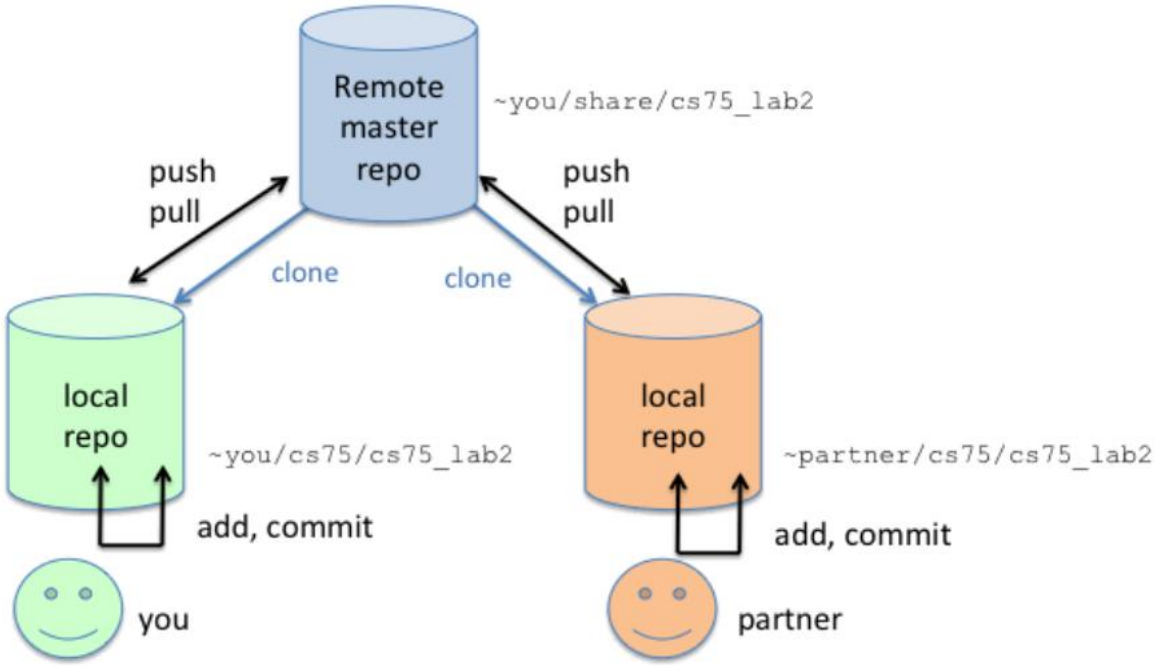
Flusso tipico per prendere una modifica al repository remoto:

FETCH: aggiorna il repo locale con le modifiche remote (ma non le rende subito disponibili)

PULL: aggiorna la working copy dal repo remoto

CLONE: aggiorna la working copy dal repo remoto per la prima volta

Come funziona la collaborazione



4. Installa github desktop

- Da file corsi o dal sito di github desktop scarica github desktop

  https://desktop.github.com/

- C'è anche una versione portable

  https://github.com/daemondevin/GitHubDesktopPortable/releases

  https://sourceforge.net/projects/xjdhdr-random-code/files/Portable_Apps/ghdPortable/ghdPortable%20v3.2.3%20Dev%20Test%202.7z/download

5. Clone del repository in locale e modifica in locale di un file di testo

- **Requisiti (cosa devi aver fatto prima)**:
  avere creato un repository (anche vuoto) e aver invitato i tuoi colleghi

- Adesso portiamo il repository in locale usiamo **CLONE**
  - Cloniamo ad esempio sul desktop o sotto documenti – chiamiamo REPO_LOCALDIR il posto dove avete fatto il clone
    Ad esempio c:\Users\garganti\miorepo
- Uno del gruppo aggiunge ad esempio sotto un folder docs un documento di testo tipo **spiegazione.txt** e mandiamolo sul repository.
- EDITING COLLABORATIVO:
  - Modifica in parallelo il file e poi il commit e push dei file modificati.
  - Nel file mettete i vostri nomi ad esempio. OGNUNO IL SUO
  - Potrebbe essere necessario fare il pull/fetch per prendere le modifiche

- ATTENTO: se modificate le stesse frase si ha errore.

- Alla fine tutti nel gruppo devono avere lo stesso file con i nomi dei partecipanti al gruppo

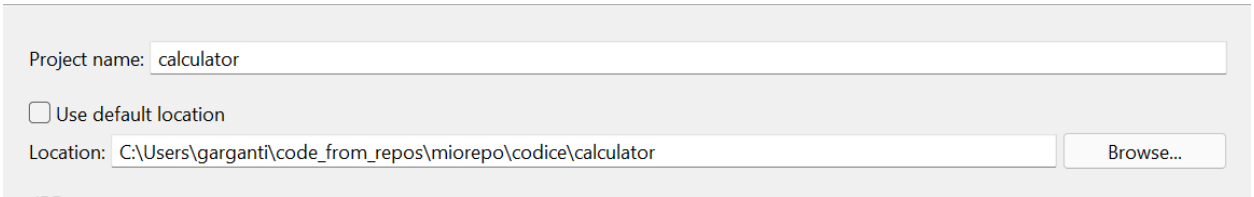
6. Lavoriamo con il codice – progetto iniziale

- **Requisiti (cosa devi aver fatto prima)**:
  avere creato un repository (anche vuoto) e aver invitato i tuoi colleghi
  aver creato un folder "codice" nel repo dove mettere il codice. Nel folder code, metti un readme.md che dice "questo folder contiene il codice".

Come caricare (pubblicare) il primo progetto (eclipse) su github (modo più semplice)

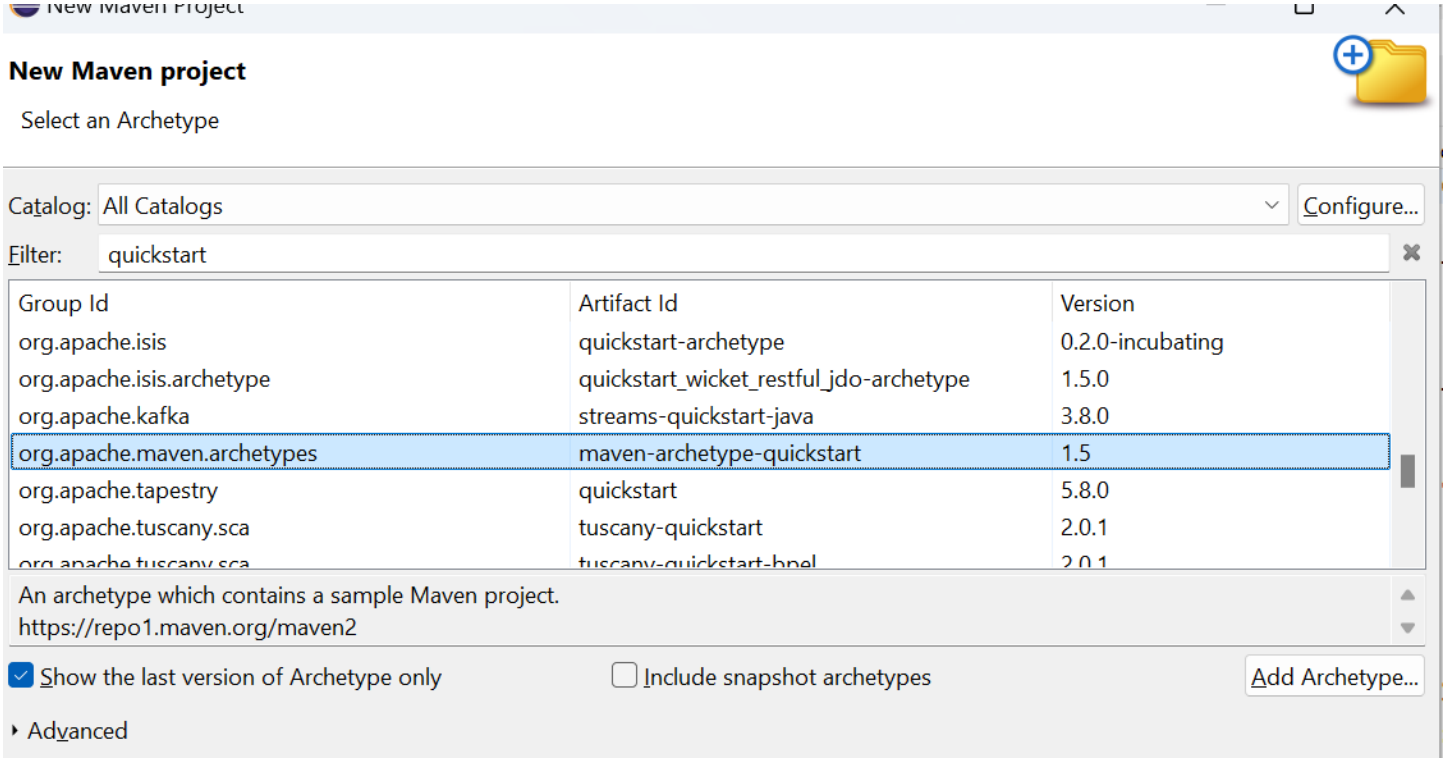
UNO SOLO MEMBRO DEL GRUPPO (gli altri assistono):

- Apri eclipse
- Prendi come workspace il desktop
- Crea un progetto (maven) che ha come root il folder codice REPO_LOCALDIR che hai scaricato localmente



Ad esempio potrebbe essere c:\Users\garganti\miorepo\codice\calculator

Attenzione: mettiamo il progetto in una sottodirectory di codice (nell'immagine si chiama calculator). In questo modo il folder codice potrà contenere più progetti.



- Con eclipse aggiungi qualche classe .java e fai il commit e push

Vedrò il mio progetto su github così:



- ALTRI COMPONENTI DEL GRUPPO:
  - **Requisito**: avete il repo clonato sul vostro computer
  - Fate il pull
  - Per aprire il progetto con eclipse dovete:
    - aprire eclipse,
    - prendete il desktop come workspace, importate il progetto dal local repo

- Alla fine tutti nel gruppo hanno il progetto eclipse sul loro pc con lo stesso codice.

7. Lavoriamo con il codice in modo collaborativo

- Scrivete la classe Calculator e un main e commitate il tutto (dividetevi i compiti, ad esempio le operazioni)
  - Un membro del team implementa in Calculator **add**, uno **prod**, uno **div** e uno **diff** Che prendono 2 numeri e fanno l'operazione
  - Un membro del gruppo scrive il main

  Ognuno push il suo metodo e gli altri faranno pull

  Alla fine tutti hanno la stessa versione di Calculator con tutte le operazioni

  Nota per ridurre la possibilità che ci siano conflitti, potresti preparare la classe Calculator prima delle modifiche così (ad esempio con l'interfaccia web):

```
1    package calcolatore.calcolatorejava;
2
3    class Calculator{
4
5
6      // lo sviluppatoe A qui ci mette l'implementazioen della somma
7
8
9      // lo sviluppatore B qui ci mette l'implementazioen della differenza
10
11
12       // lo sviluppatore C qui ci mette l'implementazioen della moltiplicazione
13
14   }
```

E poi i tre sviluppatori procedono nell'area a loro assegnata.

8. Conflitti

Potrebbero esserci dei conflitti che vanno risolti

Il modo più rapido è usare un editor testuale o eclipse stesso, rimuovere i conflitti (<<< .. HEAD …) e rimettere il tutto sul repo